Signal Handling.

By ,
Mr. GOPI

Signal

A signal is a notification to a process.

Generally signals interrupt the normal flow of execution of a program

Signal may be received either synchronously or asynchronously,

Synchronous signals are delivered to the same process that performed the operation that caused the signal.



When a signal is generated by an event external to a running process, that process receives the signal **asynchronously.**

The following small code tries to divide by zero in the program. Termination by floating point exception signal (SIGFPE). This signal occurred **synchronously**.

```
int main()
{
int a=5,b=0,c;
c=a/b;
printf("%d\n",c);
}
$ ./a.out
Floating point exception ( core dumped )
$
```

When you press ^C (ctrl+c) for infinitely running program, the program terminates on reception of Terminal interrupt (SIGINT). This is **asynchronous** signal.

```
main()
{
printf("Hi...\n");
while(1);
}
$ ./a.out
Hi...
^C
$
```

Source of Signal

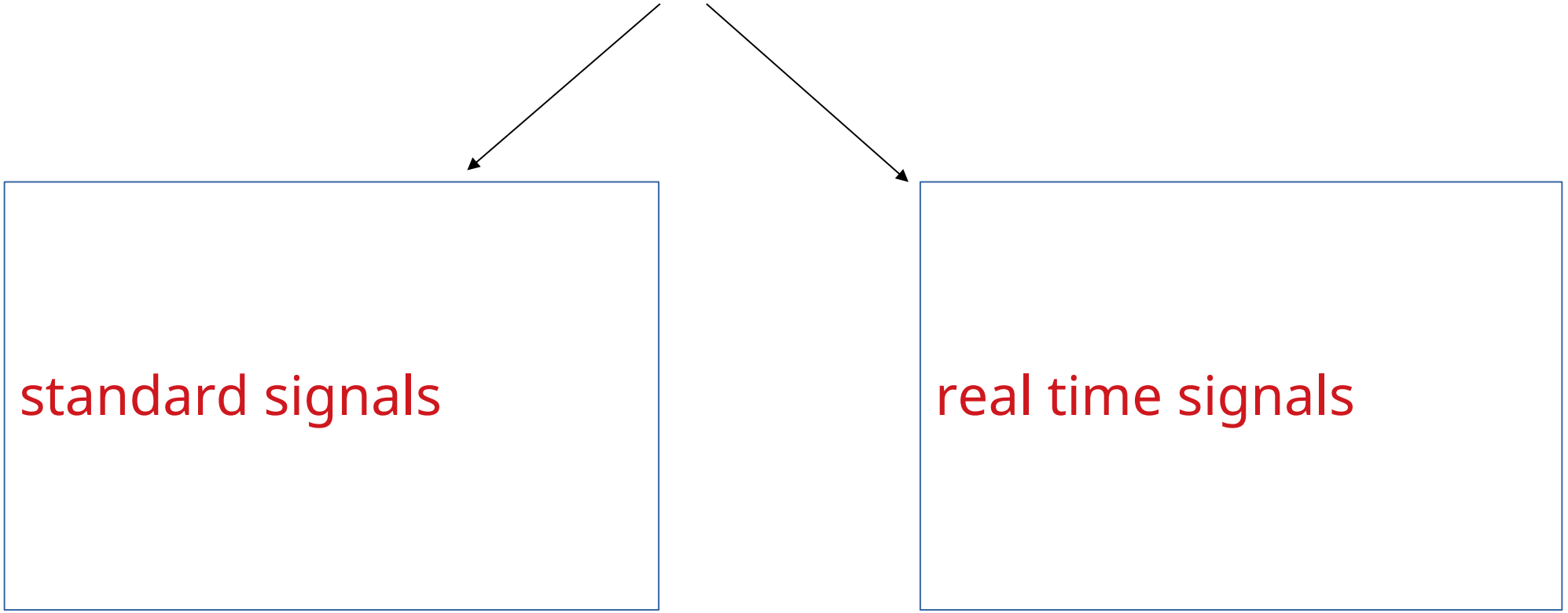
From other subsystem:- A hardware exception occurred

From the terminal:- The user typed one of the terminal special characters that generate signals

From the kernel:- A software event occurred

Signal numbers

Signals fall into two broad categories.



- Every signal have some unique number
- The standard signals are numbered from 1 to 31.
- real time signals are numbered from 32 to 63

You can list the signals on your system with command

$ kill -l

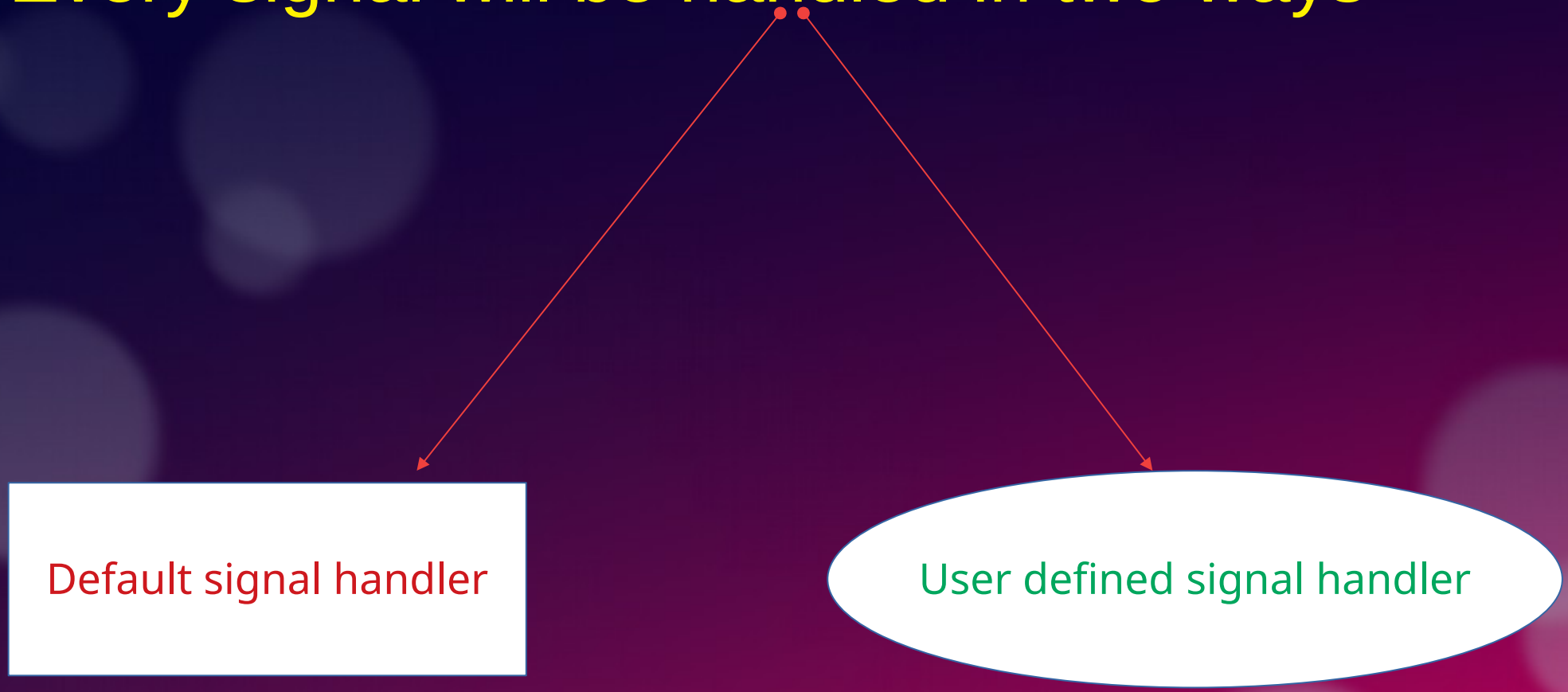
All signals, whether synchronous or asynchronous, follow the same pattern.

- $ A signal is said to be generated by some event.
- $ Once generated, later it delivered to a process.
- $ process then takes some action in response to the signal

Between the time it is generated and the time it is delivered, a signal is said to be ***pending***.

The life of signal starts when signal is generated and its life ends when signal manager delivers it to respective process.

Every signal will be handled in two ways

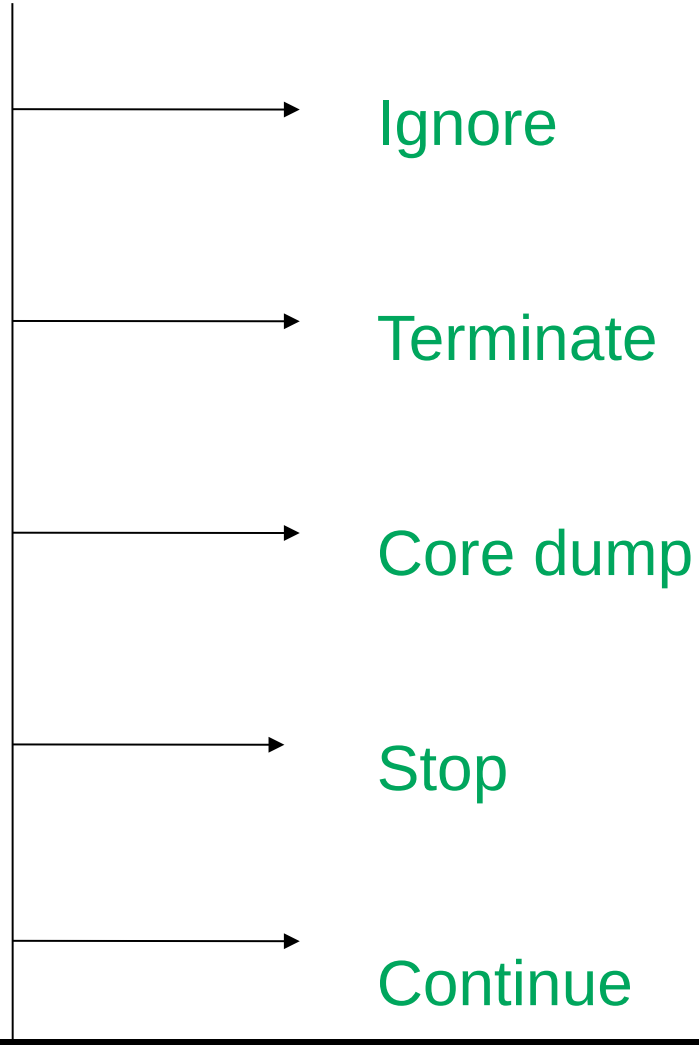


Handling signal

Every signal has a default handler that will run by the KERNEL , this action can be consider as a default signal handler.

Default action can be overriden by user-defined signal handler. This is called handling the signal.

Upon delivery of signal , a process carries one of the following default acton, depending on signal



IGNORE means , the signal never ever knows this is occured.

TERMINATE means , the process killed.

CORE-DUMP means , may be because , memory violation, the process has terminated.

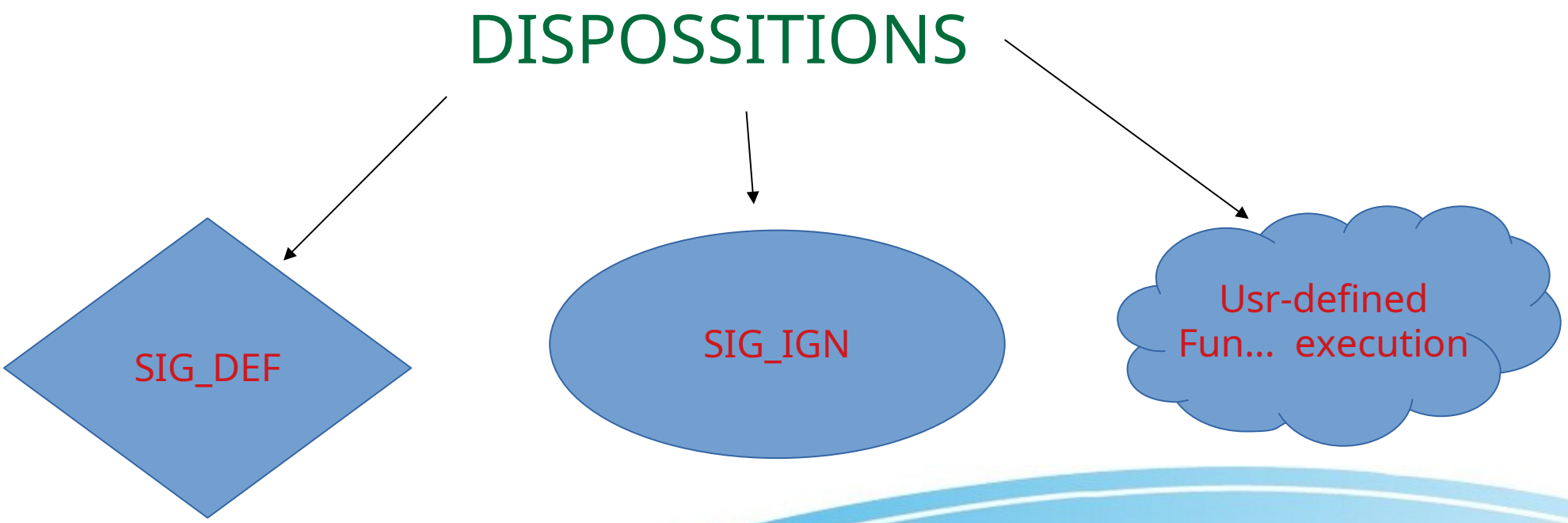
STOP means , execution of process suspended.

CONTINUE means , execution of process resumed after the process stopped .

Instead of accepting the default action , a program can change the action when signal is delivered.

This is know as setting dispossition of the signal.

We can set one of following dispossition



SIG_DFL: The default action should occur.
This is useful to undo an earlier change of the disposition of the signal.

SIG_IGN: The signal is ignored.
This is useful for a signal whose default action would be to terminate the process.

A signal handler is a function, written by the programmer that performs appropriate tasks in response to the delivery of a signal.

Notifying the kernel that a handler function should be invoked is usually referred to as establishing a signal handler.

When a signal handler is invoked in response to the delivery of a signal, you can say that the signal has been handled.

Can we handle all signals ? ------>NO

The signals SIGKILL and SIGSTOP can’t be Blocked, ignored, or caught by a handler.

When your application is itself unresponsive, there should be some way to exit the application.

As your task cannot handle anything at this state, there should be a way signal is sent to the unresponsive task but is not handled by the unresponsive task itself.

Sending and Receiving signals

To send signal from one process to another process
We can use a system call kill().

As-well, we can send signal to the same process by
Calling lib function raise().

We can make our process to sleep untill signal arrives
By using system call pause().

Syatem call alarm() is use to raise SIGALRM signal

Few system calls list

```
int pause(void);

unsigned int alarm(unsigned int seconds);

int kill(pid_t pid, int sig);

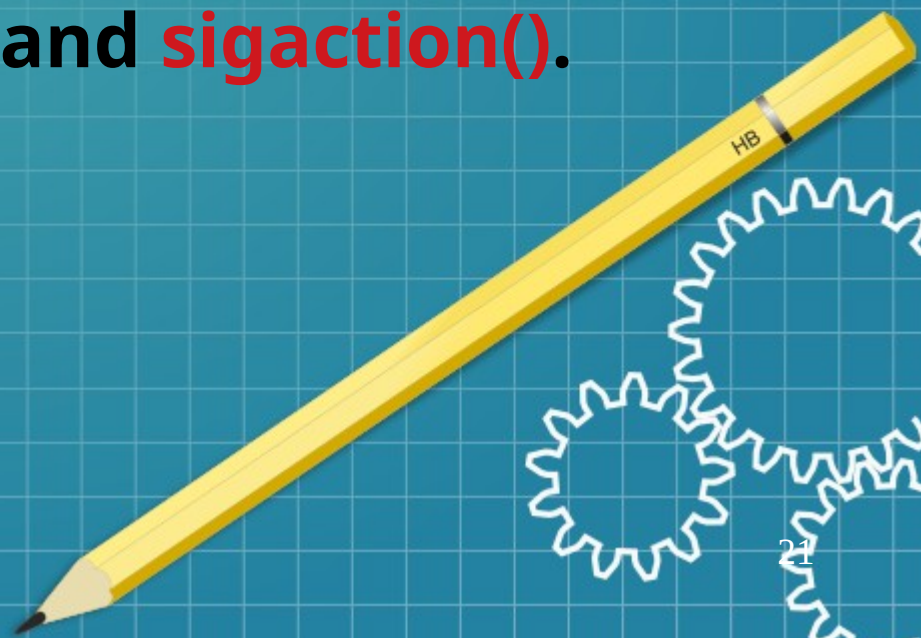
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);

int sigaction(int signum, const struct sigaction *act,
                          struct sigaction *oldact);
```

As explained previously the default disposition for each signal are terminate, ignore, core dump, stop or continue.

A process can change the disposition of a signal.

UNIX systems provide two ways of changing the disposition of a signal: signal() and sigaction().



Using these system calls, a process can elect one of the following behaviors to occur on delivery of the signal: perform the default action , ignore the signal or catch the signal.

with a signal handler, a usr- defined function that is automatically invoked when the signal is delivered.



signal

Signal is a syatem call

Returns previous signal disposition on success, or SIG_ERR on error.

The first argument , signum, identifies the signal whose disposition we wish to change

The second argument, handler, is the address of the function that should be called when this signal is delivered

sigaction

Signal is a syatem call.

Returns 0 on success or -1 on error.

The sig argument identifies the signal whose disposition we want to retrieve or change.

This argument can be any signal except SIGKILL or SIGSTOP.

Sigaction can use in 3 ways:

sigaction(signum,&v1,0);

Setting new action but not collecting old action

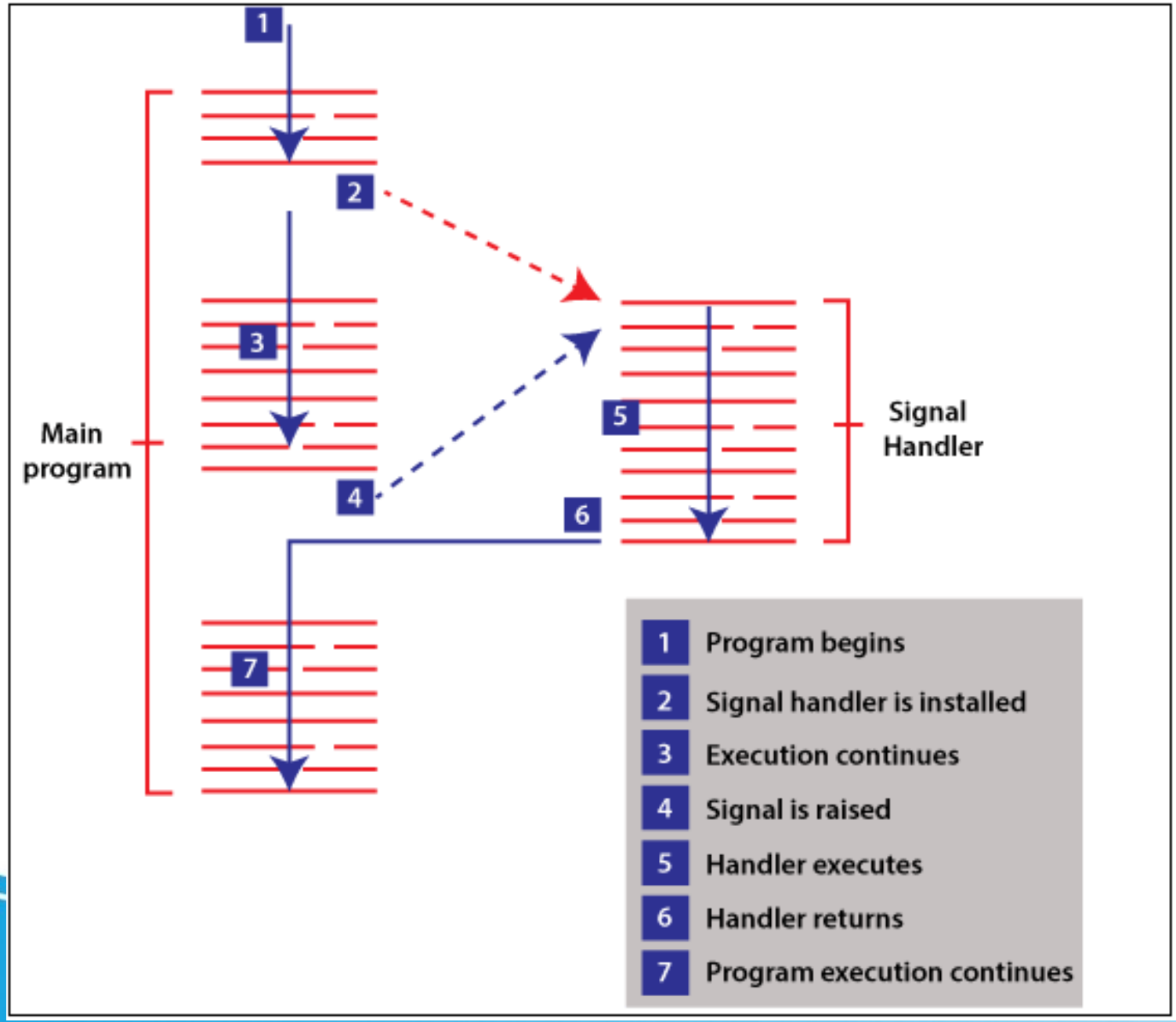
sigaction(signum,0, &v2);

Collecting old action but not setting new action

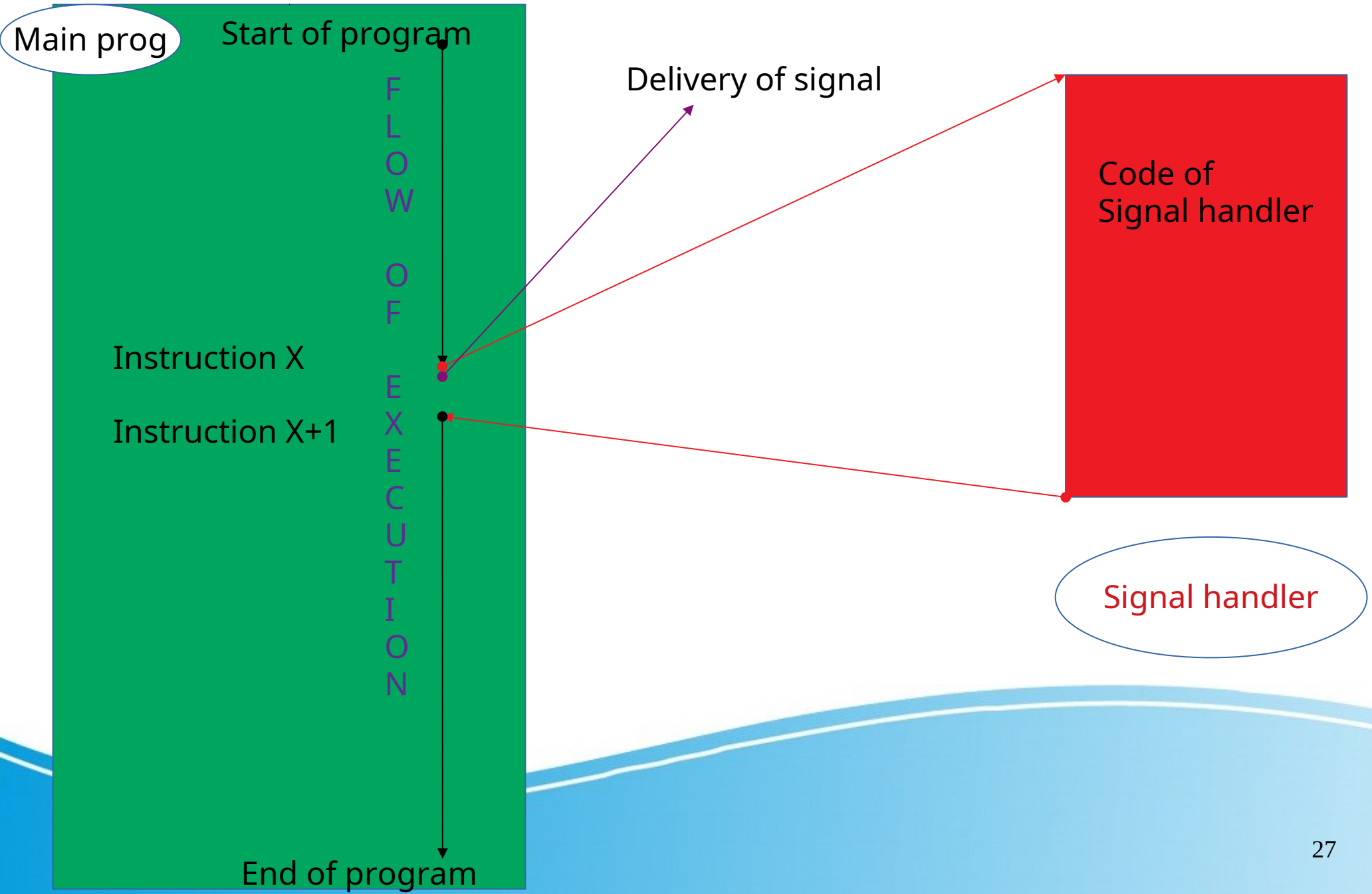
sigaction(signum,&V1, &v2);

Setting new action and collecting old action

Approach of writing program to handle a signal



Execution of program when signal is delivered



THANK YOU

